

SmartInfoHub – Android Application Documentation

1. Project Description

SmartInfoHub is an Android application developed as a demonstration of modern mobile development practices using Android Studio and Kotlin. The application showcases essential components such as fragment-based navigation, user interface interaction with various widgets, data visualization from a local file, and inter-application communication via Intents.

The primary purpose of SmartInfoHub is educational. It is designed to be used in academic environments, such as mobile programming labs or Android development courses, to illustrate how to structure and implement a multi-screen Android application using clean architectural principles and Android Jetpack components.

The application contains three distinct screens, each offering different types of interactions and visual information. It integrates Android features such as BottomNavigationView, Fragments, ViewBinding, RecyclerView, and runtime data handling from asset files.

2. User Manual

2.1 Home Screen

The home screen acts as the entry point of the application and includes two main interactive components:

- **"Choose Option" Button:**
When tapped, this button opens a dialog box presenting the user with three options: "Red", "Green", and "Blue". Upon selecting an option, a toast message is displayed indicating the selected color. This functionality demonstrates the use of AlertDialog with a list of choices.
- **"Open Google" Button:**
This button launches the default web browser and navigates to "<https://www.google.com>" using an implicit Intent. This showcases how the application can start another application through the Android Intent system.

Associated code:

- Layout: `fragment_home.xml`
- Logic: `HomeFragment.kt`
- Components: `AlertDialog`, `Toast`, `Intent.ACTION_VIEW`

2.2 Info Screen

The info screen contains interactive UI elements that demonstrate event handling beyond basic button clicks:

- **Theme Toggle (Switch):**
This switch allows the user to toggle between dark and light themes (placeholder logic). The toggle event is captured using `setOnCheckedChangeListener`.
- **Volume Control (SeekBar):**
A seek bar is provided for adjusting a hypothetical volume level. As the user changes the slider, a text field updates dynamically to reflect the current value using `setOnSeekBarChangeListener`.

Associated code:

- Layout: `fragment_info.xml`
- Logic: `InfoFragment.kt`
- Components: `Switch`, `SeekBar`, `TextView`

2.3 Settings Screen

The settings screen demonstrates data visualization using a list and dynamic file loading:

- **Item List (RecyclerView):**
A vertical list of items is displayed using a `RecyclerView`. Each item shows a title and description.
- **Data Source:**
Data is read at runtime from a local file named `data.txt` stored in the `assets/` folder. Each line in the file is formatted as "Title|Description". The file is parsed line-by-line, and the resulting data is bound to the `RecyclerView` via a custom adapter.

Associated code:

- Layout: `fragment_settings.xml`
- Logic: `SettingsFragment.kt`
- Adapter: `ItemAdapter.kt`
- Model: `Item.kt`
- File: `app/src/main/assets/data.txt`